

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – PSEUDOCODE WRITING TASK

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 – Naturalization (BL4, BL6)	Assessment:	Assessment Tool 1 – Pseudocode Writing Task
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	N.SATHWIK	Reg. No.:	192525077
Section / Batch:	2/2025	Date:	28/08/2026

Code of Conduct

I N.SATHWIK(192525077) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.SATHWIK

Instructions

- This test contains 5 Pseudocode Writing Task questions. Total: 100 Marks.
- When a student answer using pseudocode writing task should evaluate based on the ability to design clear, logical, and efficient pseudocode solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

Section / Topic		Focus	Q Nos	Marks
Minimum Spanning Tree		Kruskal's Algorithm	1	20
Graph Sorting		Topological Sort	2	20
Shortest Path Algorithm		MST & Dijkstra's Algorithm	3	20
Shortest Path Algorithm		Dijkstra's Algorithm	4	20
Minimum Spanning Tree		Prim's or Kruskal's Algorithm	5	20
GRAND TOTAL:			100 Marks	

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

My Assessments

Course: **CSA0309RB**

PSEUDOCODE WRITING TASK

OPEN

✓ Completed

Review →

CO5_AT1_PSEUDOCODE WRITING TASK

 5/5 answered

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm
20 marks



Q2

Graph - Topological Sort
20 marks



Q3

Shortest Path Algorithm - MST
& Dijkstra's Algorithm
20 marks



Q4

Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks



Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks



5/5 answered

Q1 Minimum Spanning Tree - Kruskal's Algorithm

20 marks

Write pseudocode for constructing an MST using Kruskal's Algorithm. A city planning department needs to connect multiple zones with underground fiber-optic cables. Each zone is represented as a vertex, and possible cable connections between zones are represented as edges with installation costs (weights).

Constraints:

- All zones must be connected
- Total installation cost must be minimized
- No redundant connections (no cycles) allowed
- The network must remain fully connected

Your Answer

KRUSKAL(G)

- MST = empty set
- Sort all edges of G in increasing order of weight
- Create a separate set for each vertex
- For each edge (u, v) in sorted edges:
 - If u and v belong to different sets:
 - Add (u, v) to MST
 - Merge the sets of u and v
 - If number of edges in MST = $V - 1$:
 - Break
- Return MST

Q1

Minimum Spanning Tree -
Kruskal's Algorithm
20 marks



Q2

Graph - Topological Sort
20 marks



Q3

Shortest Path Algorithm - MST
& Dijkstra's Algorithm
20 marks



Q4

Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks



Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks



5/5 answered

A DAG represents task dependencies. Write pseudocode for Topological Sort using DFS with cycle detection.

Constraints:

- Detect cycle before sorting
- Use DFS approach

Your Answer

TOPOLOGICAL_SORT(G)

- Create visited[] and recursionStack[] arrays
- Set all values of visited[] and recursionStack[] to FALSE
- Create an empty stack
- For each vertex v in G:
 - If visited[v] = FALSE:
 - If DFS(v) = TRUE:
 - Print "Cycle detected - Topological sort not possible"
 - Return
- Print vertices from the stack

DFS(v)

- visited[v] = TRUE
- recursionStack[v] = TRUE
- For each adjacent vertex u of v:
 - If visited[u] = FALSE:
 - If DFS(u) = TRUE:
 - Return TRUE
 - Else if recursionStack[u] = TRUE:
 - Return TRUE
- recursionStack[v] = FALSE
- Push v into stack
- Return FALSE

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm
20 marks



Q2

Graph - Topological Sort
20 marks



Q3

Shortest Path Algorithm - MST
& Dijkstra's Algorithm
20 marks



Q4

Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks



Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks



5/5 answered

Q3 Shortest Path Algorithm - MST & Dijkstra's Algorithm

20 marks

Write pseudocode integrating MST + Dijkstra for Smart transportation system.

Constraints:

- Use MST for infrastructure
- Use shortest path for routing

Your Answer

SMART_TRANSPORTATION(G)

- MST = KRUSKAL(G)
- Create a new graph T using the edges in MST
- Select source vertex S
- Set distance[S] = 0
- Set distance of all other vertices = INFINITY
- Mark all vertices as unvisited
- Repeat until all vertices are visited:
 Select the unvisited vertex u with minimum distance
 Mark u as visited

 For each neighbor v of u:
 If distance[u] + weight(u,v) < distance[v]:
 distance[v] = distance[u] + weight(u,v)
 Update previous[v]
- Print the shortest path and minimum distance
- Return the result

Save Answer

Next →

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm

20 marks



Q2

Graph - Topological Sort

20 marks



Q3

Shortest Path Algorithm - MST
& Dijkstra's Algorithm

20 marks



Q4

Shortest Path Algorithm -
Dijkstra's Algorithm

20 marks



Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm

20 marks



5/5 answered

Q4 Shortest Path Algorithm - Dijkstra's Algorithm

20 marks

Shortest Path with "Toll-Free" Voucher: You have a voucher that allows you to traverse one edge for free (weight = 0). Write pseudocode to find the shortest path from S to D utilizing this voucher optimally.

Your Answer

DIJKSTRA_WITH_VOUCHER(G, S, D)

1. Set $\text{distance}[v][0] = \text{INFINITY}$ for all vertices v
2. Set $\text{distance}[v][1] = \text{INFINITY}$ for all vertices v
3. $\text{distance}[S][0] = 0$
4. Insert $(S, 0)$ into priority queue
5. While priority queue is not empty:
 Remove vertex (u, used) with minimum distance

 For each edge (u, v) with cost w :

 If $\text{distance}[u][\text{used}] + w < \text{distance}[v][\text{used}]$:
 $\text{distance}[v][\text{used}] =$
 $\text{distance}[u][\text{used}] + w$
 Insert (v, used) into queue

 If $\text{used} = 0$:
 If $\text{distance}[u][0] < \text{distance}[v][1]$:
 $\text{distance}[v][1] = \text{distance}[u][0]$
 Insert $(v, 1)$ into queue
6. Answer = $\text{minimum}(\text{distance}[D][0], \text{distance}[D][1])$
7. Print the shortest path from S to D
8. Print the minimum cost

Save Answer

Next →

QUESTIONS

Q1

Minimum Spanning Tree -
Kruskal's Algorithm
20 marks



Q2

Graph - Topological Sort
20 marks



Q3

Shortest Path Algorithm - MST
& Dijkstra's Algorithm
20 marks



Q4

Shortest Path Algorithm -
Dijkstra's Algorithm
20 marks



Q5

Minimum Spanning Tree -
Prim's or Kruskal's Algorithm
20 marks



5/5 answered

Q5 Minimum Spanning Tree - Prim's or Kruskal's Algorithm

20 marks

Second Best Spanning Tree: Design pseudocode to find the "Second Best" MST. (20 Marks)

Hint: Find the MST first, then for every edge in the MST, try removing it and finding the next best edge to replace it.

Your Answer

SECOND_BEST_MST(G)

1. Find the MST of graph G using Kruskal's Algorithm
2. Store the MST and its total cost
3. Set secondBest = INFINITY

4. For each edge e in the MST:
 Remove edge e from the MST

Find the minimum cost edge that can reconnect
the two disconnected parts

Add this edge to the MST

Calculate the new total cost

If new total cost < secondBest:
 secondBest = new total cost

Remove the replacement edge

Add edge e back to the MST

5. Return secondBest
6. Print the Second Best MST and its cost

 Save Answer